

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO BÀI TẬP LỚN
ĐẶC TẢ NGÔN NGỮ LẬP TRÌNH

NGUYÊN LÝ NGÔN NGỮ LẬP TRÌNH

Giảng viên: PGS. TS. Võ Đình Hiếu

Nhóm:
Hồ Thu Giang (25025060)
Nguyễn Thị Ngọc Minh (25025080)

Năm 2026

MỤC LỤC

1	Giới thiệu	3
2	Mục tiêu	3
3	Đặc tả từ vựng (Lexical Specification)	3
3.1	Từ khóa (Keywords)	3
3.2	Định danh (Identifier)	4
3.3	Hằng số (Literals)	4
3.4	Toán tử (Operators)	4
3.5	Ký hiệu phân tách (Separators)	4
3.6	Chú thích (Comments)	4
4	Đặc tả cú pháp (Syntax Specification)	5
4.1	Cấu trúc tổng quát chương trình	5
4.2	Khai báo biến	5
4.3	Câu lệnh gán	5
4.4	Câu lệnh điều kiện	5
4.5	Vòng lặp	6
4.5.1	Do – while	6
4.5.2	For	6
4.6	Lệnh in	7
4.7	Biểu thức	7
4.8	Độ ưu tiên toán tử	7
5	Văn phạm (Grammar – BNF)	8
6	Phân tích cú pháp (Parser & AST)	9
6.1	Cơ chế hoạt động của Recursive Descent Parser	9
6.2	Ánh xạ từ Code sang AST (Ví dụ: $y = x + 2$)	9
7	Đặc tả ngữ nghĩa (Semantic Specification)	10
7.1	Bảng ký hiệu (Symbol Table)	10
7.1.1	Cách hoạt động	10
7.1.2	Ánh xạ JVM	11
7.2	Phạm vi (Scope)	11
7.3	Kiểm tra khai báo	11
7.4	Kiểm tra kiểu dữ liệu	11
7.5	Suy luận kiểu (Type Inference)	11
7.6	Lỗi ngữ nghĩa	11
8	Sinh mã Bytecode	12
8.1	Cấu trúc file JVM assembly	12
8.2	Local variable slot	13
8.3	Lệnh nạp hằng	13
8.4	Lệnh load và store biến	14
8.5	Lệnh toán học	15
8.6	Lệnh so sánh và logic	16
8.7	Lệnh print	18

8.8	Lệnh input	18
8.9	Lệnh điều kiện if/elseif/else	19
8.10	Lệnh vòng lặp for	20
8.11	Lệnh do-while	21
9	Xử lý lỗi (Error Handling)	21
9.1	Lỗi từ vựng	21
9.2	Lỗi cú pháp	22
9.3	Lỗi ngữ nghĩa	23
9.4	Định dạng thông báo lỗi	23
9.5	Ảnh hưởng của lỗi đến quá trình biên dịch	24
10	Quy trình xây dựng trình biên dịch và điểm cải tiến	24
11	Ví dụ full pipeline	25
11.1	Source	25
11.2	Token	25
11.3	AST	25
11.4	JVM	25
12	Kết luận	26

1 Giới thiệu

Ngôn ngữ GMLang được xây dựng dựa trên ngôn ngữ UPL trong đề bài, với mục tiêu mở rộng khả năng biểu đạt và hỗ trợ tốt hơn cho việc xây dựng trình biên dịch.

GMLang vẫn giữ các đặc điểm cốt lõi của UPL như cú pháp đơn giản, số lượng kiểu dữ liệu hạn chế và các cấu trúc điều khiển cơ bản, nhưng được cải tiến thêm một số tính năng như:

- Suy luận kiểu (type inference): GMLang cho phép sử dụng từ khóa auto để trình biên dịch tự động suy ra kiểu của biến dựa trên giá trị khởi tạo.
- Thêm các kiểu dữ liệu: Trong khi UPL chỉ yêu cầu 2 kiểu là int và bool, GMLang đã bổ sung thêm kiểu float (số thập phân) và string (chuỗi ký tự), giúp mở rộng khả năng biểu đạt của ngôn ngữ.
- Cải tiến hệ tập toán tử và độ ưu tiên: GMLang bổ sung thêm các toán tử logic &&, ||, ! và toán tử so sánh >, >=, ==, <=, < và mở rộng độ ưu tiên để hỗ trợ biểu thức phức tạp hơn.
- Cấu trúc if linh hoạt hơn: UPL chỉ yêu cầu if then và if then else. GMLang bổ sung thêm cấu trúc elseif, cho phép kiểm tra nhiều nhánh điều kiện một cách gọn gàng hơn thay vì phải lồng nhiều lệnh if.
- Chi tiết hóa vòng lặp for: Yêu cầu của UPL về for khá chung chung. GMLang đã làm rõ cấu trúc for trong đó bao gồm cả cơ chế kiểm tra lỗi ngữ nghĩa liên quan đến biến điều khiển, start, end và step.
- Lệnh nhập liệu: GMLang bổ sung thêm lệnh input để nhận dữ liệu từ người dùng, trong khi UPL chỉ đề cập đến lệnh print.
- Sinh Java Bytecode để thực thi trên máy ảo Java (JVM)

Mục tiêu của bài là xây dựng một pipeline biên dịch đầy đủ từ mã nguồn GMLang đến bytecode, giúp minh họa rõ ràng cách hoạt động của một trình biên dịch.

2 Mục tiêu

Thiết kế ngôn ngữ GMLang với cú pháp và ngữ nghĩa rõ ràng

Xây dựng các thành phần của trình biên dịch:

- Phân tích từ vựng (Lexer)
 - Phân tích cú pháp (Parser)
 - Phân tích ngữ nghĩa (Semantic Analysis)
 - Sinh bytecode (Code Generation) và thực thi trên máy ảo Java (JVM)
-

3 Đặc tả từ vựng (Lexical Specification)

Ngôn ngữ GMLang bao gồm các loại token chính như sau:

3.1 Từ khóa (Keywords)

Các từ khóa được định nghĩa trước và không thể sử dụng làm tên biến, bao gồm:

```
begin, end, int, bool, float, string,  
auto, if, then, else, elseif, do,  
while, for, to, print, input
```

3.2 Định danh (Identifier)

```
IDENTIFIER = [a-zA-Z]+[0-9]*
```

Định danh được dùng để đặt tên biến và phải tuân theo quy tắc:

- Bắt đầu bằng ký tự chữ cái
- Có thể chứa các chữ cái tiếp theo
- Nếu có chữ số, chỉ được xuất hiện ở cuối

Ví dụ hợp lệ:

```
x, abc, var123
```

Ví dụ không hợp lệ:

```
1abc, ab1c
```

3.3 Hằng số (Literals)

Ngôn ngữ hỗ trợ hai loại hằng:

- Số nguyên: 0, 1, 100,...
- Giá trị logic: true, false
- Số thập phân: 0.1, 0.02,...
- Chuỗi: abc, xyz,...

3.4 Toán tử (Operators)

GMLang hỗ trợ các toán tử:

- Toán học: +, *, -, /
- So sánh: >, >=, ==, <=, <
- Logic: &&, ||, !
- Gán: =

3.5 Ký hiệu phân tách (Separators)

Các ký hiệu phân tách gồm:

```
; ( ) { }
```

3.6 Chú thích (Comments)

Ngôn ngữ hỗ trợ hai dạng chú thích:

```
// chú thích một dòng
/* chú thích nhiều dòng */
```

4 Đặc tả cú pháp (Syntax Specification)

4.1 Cấu trúc tổng quát chương trình

```
begin
{
  <khai báo và câu lệnh>
}
end
```

4.2 Khai báo biến

Biến phải được khai báo trước khi sử dụng. GMLang hỗ trợ ba kiểu khai báo:

- Khai báo thông thường:

```
int x;
bool a;
```

- Khai báo có khởi tạo:

```
int y = 5;
```

- Khai báo sử dụng suy luận kiểu:

```
auto z = 10;
auto b = true;
```

4.3 Câu lệnh gán

Câu lệnh gán có dạng:

```
x = 10;
y = x + 2;
```

4.4 Câu lệnh điều kiện

GMLang hỗ trợ ba dạng câu lệnh điều kiện:

- If - then

```
if (x > 5) then {
  print(x);
}
```

- If - then - else:

```
if (x > 5) then {
    print(x);
} else {
    print(0);
}
```

- If - then - elseif (có thể nhiều lần) - else:

```
if (x > 5) then {
    print(x);
} elseif (x > 3) {
    print(1);
} else {
    print (2);
}
```

GMLang hỗ trợ thêm cấu trúc elseif nhằm biểu diễn nhiều nhánh điều kiện một cách ngắn gọn. Về bản chất, elseif được chuyển đổi nội bộ thành các câu lệnh if lồng nhau trong quá trình sinh AST.

4.5 Vòng lặp

Ngôn ngữ hỗ trợ hai loại vòng lặp:

4.5.1 Do – while

```
do {
    x = x + 1;
} while (x > 5);
```

4.5.2 For

```
for (x = 0 to 10 step 1) {
    print(x);
}
```

Vòng lặp for có dạng:

```
for (x = start to end step stepValue) {
    <statement>
}
```

Trong đó x, start, end và stepValue đều phải có kiểu int.

Cách thực thi:

- Gán giá trị ban đầu start cho biến điều khiển x.
- Nếu stepValue > 0, vòng lặp tiếp tục khi x <= end.
- Nếu stepValue < 0, vòng lặp tiếp tục khi x >= end.
- Sau mỗi lần lặp, biến điều khiển được cập nhật: x = x + stepValue.
- stepValue = 0 là lỗi ngữ nghĩa vì có thể gây vòng lặp vô hạn.

Ví dụ vòng lặp tăng:

```
for (x = 0 to 3 step 1) {  
    print(x);  
}
```

Kết quả:

```
0  
1  
2  
3
```

Ví dụ vòng lặp giảm:

```
for (x = 3 to 0 step -1) {  
    print(x);  
}
```

Kết quả:

```
3  
2  
1  
0
```

Nếu hướng đi của `stepValue` không phù hợp với `start` và `end`, vòng lặp không thực hiện lần nào. Ví dụ:

```
for (x = 0 to 10 step -1) {  
    print(x);  
}
```

Trong trường hợp này, `stepValue < 0` nên điều kiện lặp là `x >= end`. Ban đầu `0 >= 10` là sai, vì vậy vòng lặp kết thúc ngay.

4.6 Lệnh in

Lệnh in có dạng:

```
print(x);  
print(x + 1);
```

4.7 Biểu thức

GMLang hỗ trợ các loại biểu thức: - Biểu thức số học: `+`, `-`, `*`, `/` - Biểu thức so sánh: `>`, `>=`, `==`, `<=` - Biểu thức logic: `&&`, `||`, `!`

Ví dụ:

```
(x + 2) * 3 > 10 && !a
```

4.8 Độ ưu tiên toán tử

Mức	Toán tử
1	!
2	*, /
3	+, -
4	>, >=, ==, <=, <
5	&&
6	

Dấu ngoặc () được sử dụng để thay đổi thứ tự ưu tiên của các phép toán.

Biểu thức nằm trong dấu ngoặc sẽ được tính trước, bất kể thứ tự ưu tiên mặc định của toán tử.

5 Văn phạm (Grammar – BNF)

```

Program ::= "begin" Block "end"

Block ::= "{" DeclList StmtList "}"

DeclList ::= Decl DeclList | epsilon
StmtList ::= Stmt StmtList | epsilon

Decl ::= Type Identifier ("=" Expr)? ";"

Type ::= "int" | "bool" | "auto" | "float" | "string"

InputStmt ::= "input" "(" Identifier ")" ";"

Stmt ::= AssignStmt
      | IfStmt
      | WhileStmt
      | ForStmt
      | PrintStmt
      | InputStmt
      | Block

AssignStmt ::= Identifier "=" Expr ";"
AssignExpr ::= Identifier "=" Expr

IfStmt ::= "if" "(" Expr ")" "then" Stmt IfTail

IfTail ::= "elseif" "(" Expr ")" Stmt ElseIfList "else" Stmt | "else" Stmt | epsilon

ElseIfList ::= "elseif" "(" Expr ")" Stmt ElseIfList | epsilon

```

```

WhileStmt ::= "do" Stmt "while" "(" Expr ")" ";"

ForStmt ::= "for" "(" Identifier "=" Expr "to" Expr "step" Expr ")" Stmt

PrintStmt ::= "print" "(" Expr ")" ";"

Expr ::= AndExpr ("||" AndExpr)*
AndExpr ::= EqualityExpr ("&&" EqualityExpr)*
EqualityExpr ::= RelExpr ("==" RelExpr)*
RelExpr ::= AddExpr (">" | ">=" | "<" | "<=") AddExpr)*
AddExpr ::= MulExpr ("+" | "-" MulExpr)*
MulExpr ::= UnaryExpr ("*" | "/" UnaryExpr)*
UnaryExpr ::= "!" UnaryExpr | "-" UnaryExpr | Primary

Literal ::= INT_LITERAL | BOOL_LITERAL | FLOAT_LITERAL | STRING_LITERAL
Primary ::= Identifier | Literal | "(" Expr ")"

```

6 Phân tích cú pháp (Parser & AST)

GMLang sử dụng phương pháp **Recursive Descent Parser** (Phân tích cú pháp đệ quy xuống) để chuyển đổi chuỗi Token thành Cây cú pháp trừu tượng (AST). Đây là trình phân tích từ trên xuống (top-down), trong đó cấu trúc logic được xây dựng dựa trên sự phối hợp giữa class `Parser` và các định nghĩa Node trong `ast.py`.

6.1 Cơ chế hoạt động của Recursive Descent Parser

Trình phân tích hoạt động bằng cách thực thi các hàm đệ quy tương ứng với các quy tắc văn phạm (BNF).

- **Cơ chế Lookahead:** Parser luôn duy trì một biến `current_token` để “nhìn trước” mã nguồn, từ đó quyết định hàm nào sẽ được gọi tiếp theo.
- **Hàm `eat()` và `primary()`:** Hàm `eat()` đóng vai trò kiểm chứng cú pháp, trong khi `primary()` là điểm dừng của đệ quy, nơi xử lý các đơn vị nhỏ nhất như hằng số (`Num`, `String`, `Bool`) hoặc biến (`Var`).
- **Phân tầng ưu tiên toán tử:** Trong `parser.py`, các hàm được gọi theo thứ tự ưu tiên từ thấp đến cao. Điều này đảm bảo các toán tử có độ ưu tiên cao sẽ nằm sâu hơn trong cây AST và được tính toán trước.

6.2 Ánh xạ từ Code sang AST (Ví dụ: $y = x + 2$)

Dựa trên cấu trúc mã nguồn bạn đã cung cấp, quá trình xây dựng AST cho câu lệnh gán diễn ra như sau:

1. **Khởi tạo (`assign_stmt`):** Parser nhận diện `y`, sau đó gọi hàm `expr()`.
2. **Phân cấp biểu thức:** `expr()` $\rightarrow$ `add_expr()`. Tại đây, Parser phát hiện dấu `+`.

3. **Xây dựng các Node con:** * Nhánh trái: Gọi `primary()` trả về đối tượng `Var(name='x')`.
 - Nhánh phải: Gọi `primary()` trả về đối tượng `Num(value=2)`.
4. **Tạo Node trung gian:** Hàm `add()` sử dụng class `BinOp` trong `ast.py` để kết hợp: `BinOp(left=Var('x'), op=TokenType.PLUS/token '+', right=Num(2))`.
5. **Hoàn thiện Node gốc:** Hàm `assign_stmt()` trả về đối tượng `Assign(name='y', expr=BinOp(...))`.

Sơ đồ cấu trúc AST:

```

    Assign (Node)
    /      \
name: "y"   expr: BinOp (Node)
                / | \
            left: Var op: + right: Num
                (x)      (2)

```

7 Đặc tả ngữ nghĩa (Semantic Specification)

7.1 Bảng ký hiệu (Symbol Table)

Bảng ký hiệu (Symbol Table) được sử dụng trong quá trình phân tích ngữ nghĩa để lưu trữ thông tin về các biến đã được khai báo trong chương trình.

Mỗi mục trong bảng ký hiệu bao gồm các thông tin sau:

Tên biến	Kiểu	Phạm vi	Index
x	int	global	1
y	int	block	2

Trong đó:

- Tên biến (Name): định danh của biến
- Kiểu dữ liệu (Type): int, bool, string, float (hoặc suy luận từ auto)
- Phạm vi (Scope): xác định theo khối { }
- Chỉ số (Index): vị trí của biến trong local variable của JVM

7.1.1 Cách hoạt động

- Khi khai báo:
 - Thêm vào Symbol Table
 - Kiểm tra trùng tên
- Khi sử dụng:
 - Tra cứu bảng

- Kiểm tra kiểu
- Khi ra khỏi block:
 - Xóa biến khỏi bảng

7.1.2 Ánh xạ JVM

Mỗi biến sẽ được ánh xạ sang một local variable slot

```
x → slot 1 → istore_1 / iload_1
y → slot 2 → istore_2 / iload_2
```

7.2 Phạm vi (Scope)

- Phạm vi của biến được xác định bởi các khối { }
- Biến chỉ có hiệu lực trong phạm vi mà nó được khai báo và các phạm vi con

7.3 Kiểm tra khai báo

- Biến phải được khai báo trước khi sử dụng
- Không được khai báo trùng tên trong cùng một phạm vi.

7.4 Kiểm tra kiểu dữ liệu

Các quy tắc kiểm tra kiểu:

- Phép +, -, *, / chỉ áp dụng cho kiểu int, float
- Phép so sánh >, >=, <, <= chỉ áp dụng cho int, float
- Phép == áp dụng cho cả int, bool, float, string
- Với kiểu string, phép == so sánh nội dung chuỗi.
- Các phép logic (&&, ||, !) chỉ áp dụng cho bool.
- Trong vòng lặp for, biến điều khiển, start, end và step phải có kiểu int; step không được bằng 0.

7.5 Suy luận kiểu (Type Inference)

Từ khóa auto cho phép trình biên dịch tự động suy ra kiểu của biến dựa trên giá trị khởi tạo.

```
auto x = 5;           // int
auto a = true;        // bool
auto b = 5.5;          // float
auto c = "hello";     // string
```

7.6 Lỗi ngữ nghĩa

Một số lỗi phổ biến:

- Gán sai kiểu:

```
int x;
x = true; // sai kiểu
```

- Sử dụng biến chưa được khai báo

```
x = 5;           // chưa khai báo
```

8 Sinh mã Bytecode

Sau khi chương trình vượt qua phân tích từ vựng, cú pháp và ngữ nghĩa, bộ sinh mã duyệt cây AST và sinh ra mã JVM assembly. Mã này được ghi ra file `.j`, sau đó assembler tối giản trong `classfile.py` chuyển thành file `.class` để chạy trên JVM.

JVM là máy ảo dạng stack-based. Điều này có nghĩa là các phép tính không thao tác trực tiếp trên biến, mà thường diễn ra qua operand stack:

```
đẩy giá trị lên stack
đẩy giá trị tiếp theo lên stack
thực hiện lệnh tính toán
lưu kết quả về local variable slot nếu cần
```

Ví dụ:

```
int x = 1 + 2;
```

có thể được sinh thành:

```
iconst_1
iconst_2
iadd
istore 1
```

Ý nghĩa:

- `iconst_1`: đẩy số nguyên 1 lên operand stack.
- `iconst_2`: đẩy số nguyên 2 lên operand stack.
- `iadd`: lấy hai số nguyên trên stack, cộng lại và đẩy kết quả lên stack.
- `istore 1`: lấy kết quả trên stack và lưu vào local variable slot 1.

8.1 Cấu trúc file JVM assembly

File `.j` sinh ra có cấu trúc chính như sau:

```
.class public Main
.super java/lang/Object

.method public static main([Ljava/lang/String;)V
.limit stack 100
.limit locals 100

; các lệnh JVM được sinh từ chương trình GMLang

return
.end method
```

Trong đó:

- `.class public Main`: khai báo class JVM tên `Main`.
- `.super java/lang/Object`: class này kế thừa từ `java.lang.Object`.
- `.method public static main([Ljava/lang/String;)V`: khai báo hàm `main` tương ứng với `public static void main(String[] args)`.
- `.limit stack 100`: giới hạn kích thước operand stack.
- `.limit locals 100`: giới hạn số local variable slot.

8.2 Local variable slot

Mỗi biến GMLang được ánh xạ sang một local variable slot của JVM. Chỉ số slot được quản lý bằng `index` trong symbol table và trong code generator.

Ví dụ:

Biến	Kiểu	Slot
x	int	1
flag	bool	2
price	float	3
name	string	4

Slot 0 thường dành cho tham số `String[] args` của hàm `main`, nên biến của GMLang bắt đầu từ slot 1.

8.3 Lệnh nạp hằng

Các hằng số và chuỗi được đẩy lên operand stack bằng các lệnh sau:

Lệnh	Ý nghĩa
<code>iconst_m1</code>	đẩy số nguyên -1
<code>iconst_0</code> đến <code>iconst_5</code>	đẩy các số nguyên nhỏ từ 0 đến 5
<code>bipush n</code>	đẩy số nguyên trong khoảng byte
<code>sipush n</code>	đẩy số nguyên trong khoảng short
<code>ldc value</code>	đẩy hằng từ constant pool, dùng cho số lớn, float hoặc string

Sự khác nhau giữa `iconst`, `bipush`, `sipush` và `ldc`:

Nhóm lệnh	Dùng khi nào	Ví dụ	Ghi chú
<code>iconst_m1</code> , <code>iconst_0</code> ... <code>iconst_5</code>	số nguyên rất nhỏ, từ -1 đến 5	<code>iconst_3</code>	ngắn và nhanh nhất, không cần tham số dài
<code>bipush n</code>	số nguyên vừa, trong khoảng -128 đến 127	<code>bipush 10</code>	dùng khi số không nằm trong nhóm <code>iconst</code>

Nhóm lệnh	Dùng khi nào	Ví dụ	Ghi chú
<code>sipush n</code>	số nguyên lớn hơn, trong khoảng -32768 đến 32767	<code>sipush 1000</code>	dùng cho số nguyên vượt quá <code>bipush</code>
<code>ldc value</code>	hằng nằm trong constant pool	<code>ldc 100000, ldc 2.5, ldc "hello"</code>	dùng cho số lớn, float và string

Nói ngắn gọn, `iconst` là cách đẩy số nguyên nhỏ trực tiếp lên stack, còn `ldc` lấy hằng từ constant pool. Vì vậy `int x = 3`; thường dùng `iconst_3`, nhưng `float y = 2.5`; hoặc `string s = "abc"`; phải dùng `ldc`.

Ví dụ:

```
iconst_3    ; int 3
ldc 2.5     ; float 2.5
ldc "GMLang" ; string "GMLang"
```

Giá trị boolean được biểu diễn bằng số nguyên:

```
false -> iconst_0
true  -> iconst_1
```

8.4 Lệnh load và store biến

Khi cần lấy giá trị biến, codegen dùng lệnh load. Khi cần gán hoặc khởi tạo biến, codegen dùng lệnh store.

Kiểu GMLang	Load	Store	Ghi chú
<code>int</code>	<code>iload slot</code>	<code>istore slot</code>	số nguyên
<code>bool</code>	<code>iload slot</code>	<code>istore slot</code>	boolean lưu bằng 0/1
<code>float</code>	<code>fload slot</code>	<code>fstore slot</code>	số thực
<code>string</code>	<code>aload slot</code>	<code>astore slot</code>	object reference

Sự khác nhau giữa load và store:

Lệnh	Chiều dữ liệu	Ý nghĩa
<code>iload 1</code>	local slot 1 -> operand stack	lấy giá trị <code>int</code> hoặc <code>bool</code> của biến ở slot 1 để đem đi tính toán/in ra
<code>istore 1</code>	operand stack -> local slot 1	lấy giá trị trên stack và lưu vào biến <code>int</code> hoặc <code>bool</code> ở slot 1
<code>fload 2</code>	local slot 2 -> operand stack	lấy giá trị <code>float</code>
<code>fstore 2</code>	operand stack -> local slot 2	lưu giá trị <code>float</code>
<code>aload 3</code>	local slot 3 -> operand stack	lấy reference của object, ví dụ <code>string</code> hoặc <code>Scanner</code>
<code>astore 3</code>	operand stack -> local slot 3	lưu reference của object

Lệnh	Chiều dữ liệu	Ý nghĩa
------	---------------	---------

Ví dụ nếu `x` có slot 1:

```
bipush 10
istore 1    ; x = 10

iload 1     ; lấy x lên stack để dùng tiếp
```

`istore` thường xuất hiện ở khai báo biến hoặc câu lệnh gán. `iload` thường xuất hiện khi biến được dùng trong biểu thức hoặc trong `print`.

Ví dụ:

```
int x = 10;
print(x);
```

có thể sinh:

```
bipush 10
istore 1

getstatic java/lang/System/out Ljava/io/PrintStream;
iload 1
invokevirtual java/io/PrintStream/println(I)V
```

8.5 Lệnh toán học

Các phép toán số học được sinh theo kiểu dữ liệu của biểu thức.

Toán tử GMLang	Int JVM	Float JVM
+	<code>iadd</code>	<code>fadd</code>
-	<code>isub</code>	<code>fsub</code>
*	<code>imul</code>	<code>fmul</code>
/	<code>idiv</code>	<code>fdiv</code>
unary -	<code>ineg</code>	<code>fneg</code>

Quy tắc đặt tên lệnh:

Tiền tố	Kiểu dữ liệu	Ví dụ
<code>i</code>	integer, dùng cho <code>int</code> và một số thao tác với <code>bool</code>	<code>iadd</code> , <code>isub</code> , <code>iload</code> , <code>istore</code>
<code>f</code>	float	<code>fadd</code> , <code>fsub</code> , <code>fload</code> , <code>fstore</code>
<code>a</code>	reference object	<code>aload</code> , <code>astore</code>

Các lệnh toán học lấy toán hạng từ stack và đẩy kết quả lại lên stack. Ví dụ:


```
iload 1
iload 2
iadd
```

Nếu slot 1 là x , slot 2 là y , đoạn trên tương ứng với biểu thức:

```
 $x + y$ 
```

Nếu một biểu thức có cả `int` và `float`, codegen sinh thêm lệnh chuyển kiểu:

```
i2f
```

Ví dụ:

```
float y = 1 + 2.5;
```

có thể sinh theo ý tưởng:

```
iconst_1
i2f
ldc 2.5
fadd
fstore 1
```

8.6 Lệnh so sánh và logic

Các phép so sánh trả về giá trị boolean, tức là 0 hoặc 1.

Với kiểu `int`, codegen dùng các lệnh nhảy so sánh:

Toán tử	Lệnh JVM
>	<code>if_icmpgt</code>
>=	<code>if_icmpge</code>
<	<code>if_icmplt</code>
<=	<code>if_icmple</code>
==	<code>if_icmpeq</code>

Các lệnh `if_icmp...` luôn lấy hai số nguyên trên stack ra để so sánh. Nếu điều kiện đúng thì nhảy tới label.

Ví dụ:

```
iload 1
iconst_5
if_icmpgt cmp_true_0
```

Đọc là:

```
lấy x từ slot 1
đẩy số 5
nếu x > 5 thì nhảy tới cmp_true_0
```

Sự khác nhau giữa các lệnh nhảy thường gặp:

Lệnh	Điều kiện nhảy	Dùng trong project
<code>if_icmpgt label</code>	nhảy nếu giá trị thứ nhất > giá trị thứ hai	so sánh <code>int</code>
<code>if_icmpge label</code>	nhảy nếu giá trị thứ nhất $\geq$ giá trị thứ hai	so sánh <code>int</code>
<code>if_icmplt label</code>	nhảy nếu giá trị thứ nhất < giá trị thứ hai	so sánh <code>int</code>
<code>if_icmple label</code>	nhảy nếu giá trị thứ nhất $\leq$ giá trị thứ hai	so sánh <code>int</code>
<code>if_icmpeq label</code>	nhảy nếu hai giá trị bằng nhau	so sánh <code>int</code> , <code>bool</code>
<code>ifeq label</code>	nhảy nếu một giá trị trên stack bằng 0	kiểm tra điều kiện sai trong <code>if</code>
<code>ifne label</code>	nhảy nếu một giá trị trên stack khác 0	lặp lại trong <code>do-while</code>
<code>goto label</code>	nhảy vô điều kiện	bỏ qua nhánh <code>else</code> hoặc quay lại đầu vòng lặp

Ví dụ:

```
print(x > 5);
```

có thể sinh theo dạng:

```
iload 1
iconst_5
if_icmpgt cmp_true_0
iconst_0
goto cmp_end_1
cmp_true_0:
iconst_1
cmp_end_1:
```

Với kiểu `float`, JVM không dùng trực tiếp `if_fcmp...`. Codegen sinh:

```
fcmpl
ifgt / ifge / iflt / ifle / ifeq
```

Với kiểu `string`, phép `==` được sinh thành lời gọi phương thức:

```
invokevirtual java/lang/String/equals(Ljava/lang/Object;)Z
```

Các phép logic được sinh như sau:

Toán tử	Lệnh JVM
<code>&&</code>	<code>iand</code>
<code> </code>	<code>ior</code>
<code>!</code>	dùng <code>ifeq</code> , <code>goto</code> và <code>label</code> để đảo 0/1

Vì boolean được biểu diễn bằng 0 và 1, phép `&&` có thể dùng `iand`, còn `||` có thể dùng `ior`.

8.7 Lệnh print

Lệnh:

```
print(expr);
```

được sinh theo mẫu:

```
getstatic java/lang/System/out Ljava/io/PrintStream;
<sinh mã cho expr>
invokevirtual java/io/PrintStream/println(<descriptor>)V
```

Descriptor phụ thuộc vào kiểu của biểu thức:

Kiểu	Descriptor
int	(I)V
bool	(Z)V
float	(F)V
string	(Ljava/lang/String;)V

Hai lệnh chính trong `print` có vai trò khác nhau:

Lệnh	Vai trò
<code>getstatic java/lang/System/out Ljava/io/PrintStream;</code>	lấy object <code>System.out</code> và đẩy lên stack
<code>invokevirtual java/io/PrintStream/println(...)V</code>	gọi phương thức <code>println</code> trên object <code>System.out</code>

Trước khi gọi `println`, stack phải có đủ:

```
System.out
giá trị cần in
```

Ví dụ:

```
print("hello");
```

sinh:

```
getstatic java/lang/System/out Ljava/io/PrintStream;
ldc "hello"
invokevirtual java/io/PrintStream/println(Ljava/lang/String;)V
```

8.8 Lệnh input

Lệnh:

```
input(x);
```

được sinh bằng cách tạo `java.util.Scanner` đọc từ `System.in`.

Phần tạo scanner:

```
new java/util/Scanner
dup
getstatic java/lang/System/in Ljava/io/InputStream;
invokespecial java/util/Scanner/<init>(Ljava/io/InputStream;)V
astore <scanner_slot>
```

Ý nghĩa từng lệnh khi tạo `Scanner`:

Lệnh	Ý nghĩa
<code>new java/util/Scanner</code> <code>dup</code>	cấp phát object <code>Scanner</code> mới nhân đôi reference trên stack để vừa gọi constructor vừa giữ object
<code>getstatic java/lang/System/in</code> <code>Ljava/io/InputStream;</code>	lấy <code>System.in</code> làm nguồn nhập
<code>invokespecial</code> <code>java/util/Scanner/<init>(Ljava/io/InputStream;)V</code>	gọi constructor <code>new Scanner(System.in)</code>
<code>astore <scanner_slot></code>	lưu object <code>Scanner</code> vào một local slot để dùng lại

Sau đó, tùy kiểu của biến, codegen gọi phương thức đọc tương ứng:

Kiểu biến	Scanner method
<code>int</code>	<code>nextInt()I</code>
<code>bool</code>	<code>nextBoolean()Z</code>
<code>float</code>	<code>nextFloat()F</code>
<code>string</code>	<code>next()Ljava/lang/String;</code>

Ví dụ với `int x; input(x);`:

```
aload <scanner_slot>
invokevirtual java/util/Scanner/nextInt()I
istore 1
```

8.9 Lệnh điều kiện `if/elseif/else`

Câu lệnh:

```
if (condition) then {
    ...
} else {
    ...
}
```

được sinh bằng label và lệnh nhảy:

```
<sinh mã condition>
ifeq if_next_0

; then branch
goto if_end_1

if_next_0:
; else branch

if_end_1:
```

Trong đó:

- `ifeq label`: nhảy nếu giá trị trên stack bằng 0, tức điều kiện sai.
- `goto label`: nhảy vô điều kiện.
- Các label như `if_next_0`, `if_end_1` đánh dấu vị trí trong mã JVM.

8.10 Lệnh vòng lặp for

Vòng lặp:

```
for (i = start to end step stepValue) {
    body
}
```

được sinh theo ý tưởng:

```
<sinh mã start>
istore <slot_i>

for_start:
<kiểm tra step dương hay âm>

<nếu step dương, kiểm tra i <= end>
<nếu step âm, kiểm tra i >= end>

for_body:
<sinh mã body>

iload <slot_i>
<sinh mã step>
iadd
istore <slot_i>
goto for_start

for_end:
```

Project hỗ trợ cả vòng lặp tăng và giảm. Vì vậy codegen kiểm tra dấu của `step`:

- Nếu `step > 0`, vòng lặp tiếp tục khi `i <= end`.

- Nếu `step < 0`, vòng lặp tiếp tục khi `i >= end`.
- Nếu `step == 0`, semantic analyzer đã báo lỗi trước khi đến codegen.

8.11 Lệnh do-while

Câu lệnh:

```
do {
    body
} while (condition);
```

được sinh theo mẫu:

```
do_start_0:
<sinh mã body>
<sinh mã condition>
ifne do_start_0
```

Trong đó:

- `ifne label`: nhảy nếu giá trị trên stack khác 0, tức điều kiện đúng.
- Vì label nằm trước body, do-while luôn chạy body ít nhất một lần.

Chương trình sau khi biên dịch sẽ được thực thi trực tiếp trên JVM.

9 Xử lý lỗi (Error Handling)

Trong GMLang, xử lý lỗi là một phần quan trọng của quá trình biên dịch. Mục tiêu không chỉ là phát hiện chương trình sai, mà còn cung cấp thông tin đủ rõ để người lập trình biết lỗi nằm ở đâu và thuộc loại nào.

Trình biên dịch phát hiện ba nhóm lỗi chính:

Loại lỗi	Giai đoạn phát hiện	Ví dụ
Lỗi từ vựng	Lexer	ký tự không hợp lệ, token không thuộc ngôn ngữ
Lỗi cú pháp	Parser	thiếu <code>;</code> , thiếu <code>}</code> , sai cấu trúc <code>if</code> , <code>for</code>
Lỗi ngữ nghĩa	Semantic Analyzer	sai kiểu, biến chưa khai báo, khai báo trùng

9.1 Lỗi từ vựng

Lỗi từ vựng xảy ra khi mã nguồn chứa ký tự hoặc chuỗi ký tự không thể tạo thành token hợp lệ của GMLang.

Ví dụ:

```
int x = 10 @;
```

Ký tự @ không thuộc tập toán tử hoặc ký hiệu phân tách của GMLang, vì vậy lexer báo lỗi.

Đặc điểm:

- Lỗi được báo kèm vị trí dòng/cột.
- Lexer thường dừng tại lỗi đầu tiên vì nếu token không hợp lệ thì parser không có đầu vào đáng tin cậy để tiếp tục.

9.2 Lỗi cú pháp

Lỗi cú pháp xảy ra khi chuỗi token không khớp với grammar của ngôn ngữ.

Ví dụ:

```
begin
{
  int x = ;
  print(x)
}
end
```

Các lỗi trong ví dụ:

- Thiếu biểu thức sau dấu =.
- Thiếu dấu ; sau lệnh `print(x)`.

Chiến lược xử lý lỗi cú pháp:

- Parser ghi nhận lỗi tại vị trí phát hiện.
- Sau khi gặp lỗi, parser thực hiện recovery bằng cách bỏ qua token cho đến khi gặp một điểm đồng bộ.
- Các điểm đồng bộ chính là ; và }.
- Sau đó parser tiếp tục phân tích phần còn lại của chương trình.

Nhờ đó, parser có thể báo nhiều lỗi cú pháp trong một lần biên dịch thay vì chỉ dừng ở lỗi đầu tiên.

Lưu ý: GMLang quy định trong một block, phần khai báo phải đứng trước phần câu lệnh:

```
Block ::= "{" DeclList StmtList "}"
```

Vì vậy đoạn sau là lỗi cú pháp:

```
begin
{
  int x;
  x = 1;
  int y;
}
end
```

Lý do là `int y;` xuất hiện sau một câu lệnh gán, trong khi grammar không cho phép quay lại phần khai báo sau khi đã bắt đầu phần câu lệnh.

9.3 Lỗi ngữ nghĩa

Lỗi ngữ nghĩa xảy ra khi chương trình đúng cú pháp nhưng vi phạm các luật về kiểu dữ liệu, khai báo hoặc phạm vi.

Ví dụ:

```
begin
{
  int x;
  bool flag = true;

  x = true;
  y = 10;
  if (x) then {
    print(x);
  }
}
end
```

Các lỗi ngữ nghĩa:

- `x = true;`: không thể gán giá trị `bool` cho biến `int`.
- `y = 10;`: biến `y` chưa được khai báo.
- `if (x)`: điều kiện `if` phải có kiểu `bool`, nhưng `x` có kiểu `int`.

Semantic analyzer kiểm tra các luật chính:

- Biến phải được khai báo trước khi sử dụng.
- Không được khai báo trùng tên trong cùng một scope.
- Kiểu của biểu thức gán phải tương thích với kiểu của biến.
- Biến `auto` phải có giá trị khởi tạo để suy luận kiểu.
- Điều kiện trong `if`, `elseif`, `while` phải có kiểu `bool`.
- Trong vòng `for`, biến điều khiển, `start`, `end`, `step` phải có kiểu `int`.
- `step` của vòng `for` không được bằng 0.

Chiến lược xử lý lỗi ngữ nghĩa:

- Semantic analyzer không dừng ngay tại lỗi đầu tiên.
- Khi phát hiện lỗi trong một khai báo hoặc câu lệnh, lỗi được lưu lại.
- Analyzer tiếp tục kiểm tra các khai báo/câu lệnh còn lại trong block nếu có thể.
- Sau khi duyệt xong, toàn bộ danh sách lỗi được báo ra cùng lúc.

9.4 Định dạng thông báo lỗi

Mỗi lỗi cần cung cấp đủ thông tin để người dùng xác định nguyên nhân:

```
<loại lỗi>: <mô tả lỗi> at <dòng>:<cột>
```

Ví dụ:

```
Cannot assign bool to int at 7:5
Undefined variable 'y' at 8:5
Condition must be bool at 10:9
```


Khi có nhiều lỗi, trình biên dịch gom các lỗi thành một danh sách:

```
error: Semantic analysis failed with 3 error(s):
Cannot assign bool to int at 7:5
Undefined variable 'y' at 8:5
Condition must be bool at 10:9
```

9.5 Ảnh hưởng của lỗi đến quá trình biên dịch

Quy trình biên dịch chỉ tiếp tục sang pha tiếp theo khi pha hiện tại thành công.

```
Lexer
-> Parser
-> Semantic Analyzer
-> Code Generator
-> Class File Writer
```

Nếu một pha phát hiện lỗi nghiêm trọng:

- Lexer lỗi: dừng trước parser.
- Parser lỗi: dừng trước semantic analyzer.
- Semantic lỗi: dừng trước code generation.
- Code generation lỗi: không tạo file .class hợp lệ.

Như vậy, file bytecode chỉ được sinh khi chương trình vượt qua đầy đủ các bước kiểm tra từ vựng, cú pháp và ngữ nghĩa.

10 Quy trình xây dựng trình biên dịch và điểm cải tiến

Ngôn ngữ GMLang không chỉ dừng lại ở việc định nghĩa cú pháp và ngữ nghĩa, mà còn được thiết kế để hỗ trợ một quy trình biên dịch hoàn chỉnh từ mã nguồn đến bytecode.

Quá trình biên dịch của GMLang được tổ chức theo các bước liên tiếp như sau:

- Đầu tiên, bộ phân tích từ vựng (Lexer) thực hiện việc đọc mã nguồn và chuyển đổi chuỗi ký tự thành các token. Giai đoạn này chịu trách nhiệm loại bỏ khoảng trắng, nhận diện từ khóa, định danh, toán tử và phát hiện các lỗi từ vựng.
- Tiếp theo, bộ phân tích cú pháp (Parser) sử dụng các token để xây dựng cây cú pháp trừu tượng (AST). AST biểu diễn cấu trúc chương trình dưới dạng cây, giúp dễ dàng xử lý ở các bước sau.
- Sau đó, bộ phân tích ngữ nghĩa (Semantic Analyzer) kiểm tra tính đúng đắn của chương trình, bao gồm kiểm tra khai báo biến, kiểm tra kiểu dữ liệu và xử lý phạm vi. Đồng thời, trong GMLang, giai đoạn này còn thực hiện suy luận kiểu cho các biến được khai báo bằng từ khóa auto.
- Sau khi hoàn thành phân tích ngữ nghĩa, trình biên dịch tiến hành sinh trực tiếp Java Bytecode từ cây AST. Mỗi node trong AST sẽ được duyệt và ánh xạ sang các lệnh tương ứng của JVM.

Source Code (GMLang)

↓

```
Lexical Analysis
↓
Syntax Analysis (AST)
↓
Semantic Analysis
↓
Java Bytecode Generation
↓
Java Bytecode (JVM)
```

Bên cạnh việc xây dựng đầy đủ pipeline compiler, GMLang còn có một số điểm cải tiến so với ngôn ngữ ban đầu:

- Thứ nhất, ngôn ngữ bổ sung cơ chế suy luận kiểu thông qua từ khóa `auto`, giúp giảm bớt việc khai báo kiểu và tăng tính linh hoạt.
- Thứ hai, hệ thống biểu thức được mở rộng với các toán tử logic như `&&`, `||`, `!`, cho phép biểu diễn các điều kiện phức tạp hơn.
- Việc sinh trực tiếp Java Bytecode giúp giảm độ phức tạp của hệ thống và tận dụng trực tiếp môi trường thực thi của JVM.
- Cuối cùng, trình biên dịch được xây dựng theo mô hình nhiều pha, tương tự các hệ thống compiler thực tế, giúp tăng tính rõ ràng, dễ bảo trì và dễ mở rộng.

11 Ví dụ full pipeline

11.1 Source

```
int x = 1 + 2;
```

11.2 Token

```
int, x, =, 1, +, 2
```

11.3 AST

```
Assign
|-- id: x
`-- Add
    |-- 1
    `-- 2
```

11.4 JVM

```
iconst_1
iconst_2
```

```
iadd  
istore_1
```

12 Kết luận

GMLang là phiên bản cải tiến của UPL với cú pháp đơn giản nhưng mạnh hơn, hỗ trợ biểu thức logic, suy luận kiểu và có thiết kế trình biên dịch đầy đủ. Ngôn ngữ này không chỉ đáp ứng yêu cầu của đề bài mà còn thể hiện rõ cấu trúc của một hệ thống compiler hiện đại, dễ mở rộng và tối ưu trong tương lai.

GMLang có thể mở rộng để hỗ trợ:

- function
- array
- optimization (constant folding)